

DATA 3464: Fundamentals of Data Processing

Wrangling text

Charlotte Curtis

March 5, 2026

Topic overview

- Text file formats and character encodings
- Extracting features from (un)structured text
- Regular expressions

Resources used:

- [Python documentation](#)
- [Joel on Software Blog Post](#)

What is a text file?

- Any file is just a sequence of **bytes**
- The file **extension** is somewhat meaningless
- Text files contain only human-readable **characters**
- **Binary files** are everything else, including:
 - Images
 - PDFs
 - Word documents*
 - Executables

Character encodings

There are three hard problems in computer science:

- 1) Converting from PDF,
- 2) Converting to PDF, and
- 3) $O_3 \rightarrow O_2 + O$

—Marseille Folog

- To properly read a text file, we need to know its **encoding**
- Character encodings define how bytes are interpreted
- Turns out this is surprisingly **complicated**

ASCII

- American Standard Code for Information Interchange (1963)
- 7-bit encoding (128 characters)
- First 32 are **control characters** (e.g., newline, tab)
- Then punctuation, digits, uppercase letters, lowercase letters
- Most computers use 8-bit bytes, so there's a whole 128 characters "left over"

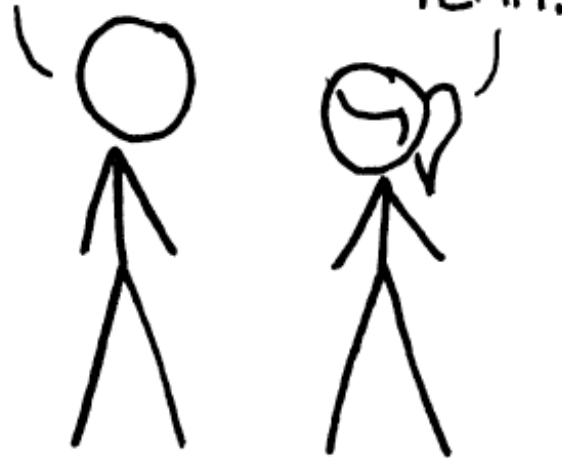
This is a very English-centric character set!

HOW STANDARDS PROLIFERATE:

(SEE: A/C CHARGERS, CHARACTER ENCODINGS, INSTANT MESSAGING, ETC.)

SITUATION:
THERE ARE
14 COMPETING
STANDARDS.

14?! RIDICULOUS!
WE NEED TO DEVELOP
ONE UNIVERSAL STANDARD
THAT COVERS EVERYONE'S
USE CASES.



SOON:

SITUATION:
THERE ARE
15 COMPETING
STANDARDS.

Unicode to the rescue

- In the 1980s things were already getting out of hand
- The **Unicode consortium** published a standard in 1993 that assigned a **code point** to every character they could think of (297,334 as of Unicode 17.0)
- Currently, the most common encoding is **UTF-8**
 - "Unicode Transformation Format – 8-bit"
 - How can 297k characters be represented in only 8 bits?
- The first 128 characters are 1 byte each and align with ASCII
- After that, **2-4 bytes per character** are used

*There's also UTF-16 and UTF-32, but now we need to deal with **endianness***

Line endings

- Say we agree to use UTF-8 👍
- A text file is still just a big long string of **bytes**
- Humans like things to be orderly and readable with **line breaks**

System	Abbreviation	Escape sequence	Code point
Pre-OS X Mac	CR (carriage return)	<code>\r</code>	U+000D
Unix	LF (line feed)	<code>\n</code>	U+000A
Windows	CRLF	<code>\r\n</code>	U+000D U+000A

- Result: chaos, but we've mostly settled on LF (`\n`)

Why am I telling you all this?

- As data scientists, you will need to ingest data from various sources
- You **will** encounter character encoding and/or line ending issues
- You don't need to memorize all this, but recognizing that an issue exists will go a long ways towards fixing it